ARIA

Agentic Research Intelligence Assistant

Plan . Retrieve . Reflect . Synthesise

Capstone Project Documentation

Version 2.0 - 10-Feature Upgrade

Tech Stack: LangGraph | ChromaDB | Groq Llama-3.3 | Streamlit
Architecture: 9-Node StateGraph | Semantic Cache | Cross-Encoder Reranker
Testing: 25 Unit Tests | 100% Pass Rate

1. Problem Statement

In the rapidly advancing field of artificial intelligence, Large Language Models (LLMs) have demonstrated exceptional capabilities in natural language understanding and generation. However, standard autonomous agents and Retrieval-Augmented Generation (RAG) paradigms suffer from several critical shortcomings that ARIA was designed to address:

1.1 Static Query Parsing

Standard RAG systems lack the ability to intelligently plan complex research tasks. They parse the user's prompt verbatim, performing a single vector similarity search rather than decomposing the question into focused, actionable sub-queries. This naive approach frequently retrieves irrelevant or superficial context, leading to shallow, incomplete answers for multi-faceted research questions.

1.2 Domain Knowledge Cutoffs

LLMs are fundamentally constrained by training data cutoffs. Without real-time access to the open web or rapidly updating repositories like arXiv, their answers degrade significantly when addressing state-of-the-art developments, recent publications, or rapidly evolving fields. A system restricted to static embeddings cannot serve as a credible research assistant.

1.3 Absence of Self-Verification

Most critically, standard systems possess no intrinsic mechanism for self-reflection or hallucination detection. When a model fabricates a fact or draws an improper conclusion from faulty retrieval, it delivers the flawed answer with absolute confidence. Users have no way to distinguish between grounded facts and hallucinated claims, eroding trust in the system's output.

1.4 The Need

There is an urgent need for an intelligently orchestrated, multi-agent system capable of: (a) planning complex research workflows through question decomposition, (b) dynamically routing between knowledge bases and live search tools, (c) evaluating its own output for faithfulness before presenting it to the user, and (d) iteratively improving its answers through structured self-reflection.

2. Solution: The PERS Framework

ARIA (Agentic Research Intelligence Assistant) resolves these limitations through the PERS framework: Plan, Execute, Reflect, and Synthesize. Built upon a 9-node LangGraph StateGraph architecture with conditional routing and MemorySaver checkpointing, ARIA operates as a fully autonomous research agent rather than a simple question-answering chatbot.

2.1 Plan Phase

The Planner Node decomposes each user question into 3-5 focused, non-overlapping sub-queries using the LLM. It autonomously determines the optimal retrieval route for each question:

- 'retrieve': Use the local ChromaDB knowledge base (established AI/ML concepts)
- 'tool': Use live arXiv and web search (recent papers, current events)
- 'both': Combine both sources (unclear intent, or question needs both)

The route decision is then validated by a KB Coverage Validator that checks the actual ChromaDB similarity score. If the best match has an L2 distance > 1.2 (poor coverage), the system overrides the LLM's route to 'both', ensuring comprehensive coverage even when the planner makes a suboptimal decision.

2.2 Execute Phase

Execution proceeds through two parallel paths depending on the route:

Knowledge Base Retrieval: The Retrieve Node queries ChromaDB with top-k=10 using the all-MiniLM-L6-v2 bi-encoder. Results from both the base collection (4,437 chunks from 45+ academic sources) and any session-specific user-uploaded documents are merged. A Cross-Encoder Reranker (ms-marco-MiniLM-L-6-v2) then rescores all candidates against the original query, selecting the top 5 most relevant chunks.

Live Search: The Tool Node executes arXiv academic search and DuckDuckGo web search on each sub-query, extracting publication metadata for research timeline visualization.

2.3 Reflect Phase

After the Answer Node generates a structured JSON report, the Eval Node scores its faithfulness (0.0-1.0) by checking whether every claim in the answer is grounded in the retrieved context. If the score falls below 0.70, the Reflect Node generates a targeted critique identifying specific hallucinated claims. The graph then routes backward to the Answer Node for rewriting, with a maximum of 2 reflection retries. This adversarial loop dramatically reduces hallucination rates.

2.4 Synthesize Phase

The final answer is structured as a JSON report containing: an executive summary, 3-5 key findings, cited sources, and 2-3 follow-up questions for continued research. The Save Node persists the result to the semantic cache, updates the user profile, and refreshes the sliding context window.

3. Nine-Node Graph Architecture

ARIA's core pipeline consists of nine specialized nodes wired into a conditional LangGraph StateGraph with MemorySaver checkpointing. Each node reads from and writes to a shared ARIAState (TypedDict with 25 fields). The graph uses conditional edges for intelligent routing.

Node 1: Memory Node

Appends the user message to chronological history. Enforces a sliding context window of the last 10 interaction pairs (20 messages) to prevent token overflow while maintaining sufficient conversational context for multi-turn research sessions.

Node 2: Cache Node

Embeds the incoming question using the same all-MiniLM-L6-v2 model as ChromaDB. Computes cosine similarity (via dot product on normalized embeddings) against every prior question in the session-level cache. If similarity exceeds 0.92, the cached report is returned instantly, completely bypassing planning, retrieval, generation, and evaluation. This provides near-zero latency for repeated or semantically identical queries.

Node 3: Planner Node

Uses the LLM to decompose the question into 3-5 sub-queries and determine the retrieval route. Includes Comparison Mode detection (triggers on 'Compare:' prefix) and KB Coverage Validation (L2 distance check that overrides weak routing decisions).

Node 4: Retrieve Node

Two-stage retrieval pipeline: bi-encoder search (top-10 from ChromaDB base + session collections) followed by cross-encoder reranking to select the top 5 most relevant chunks. In Comparison Mode, runs separate queries per concept for structured analysis.

Node 5: Tool Node

Executes arXiv and DuckDuckGo searches on up to 3 sub-queries. Extracts publication date metadata from arXiv results for research timeline chart generation.

Node 6: Answer Node

Synthesizes a structured JSON report using all available context. Supports both standard mode (summary, key_findings, sources, follow_ups) and comparison mode (comparison_table, recommendation). Integrates automatic LLM failover from Groq to Gemini Flash.

Node 7: Eval Node

Scores faithfulness on a 0.0-1.0 scale against retrieved context using a detached evaluator prompt. Routes to save_node if score >= 0.70 or retries exhausted.

Node 8: Reflect Node

Generates targeted critique identifying specific hallucinated claims when faithfulness is low. Feeds the critique back to answer_node as a reflection_note for rewriting.

Node 9: Save Node

Persists the final answer to message history and semantic cache. Updates the cross-session user profile. Resets all per-turn fields for the next invocation.

4. Interface & Features

4.1 Clean Launch State

The ARIA interface uses a premium dark glassmorphism design with Inter typography (Google Fonts). The sidebar contains: user profile name/ID input, KB status (Online with chunk count), turn counter, document uploader (drag-and-drop PDF/TXT), session export button, and New Session control.

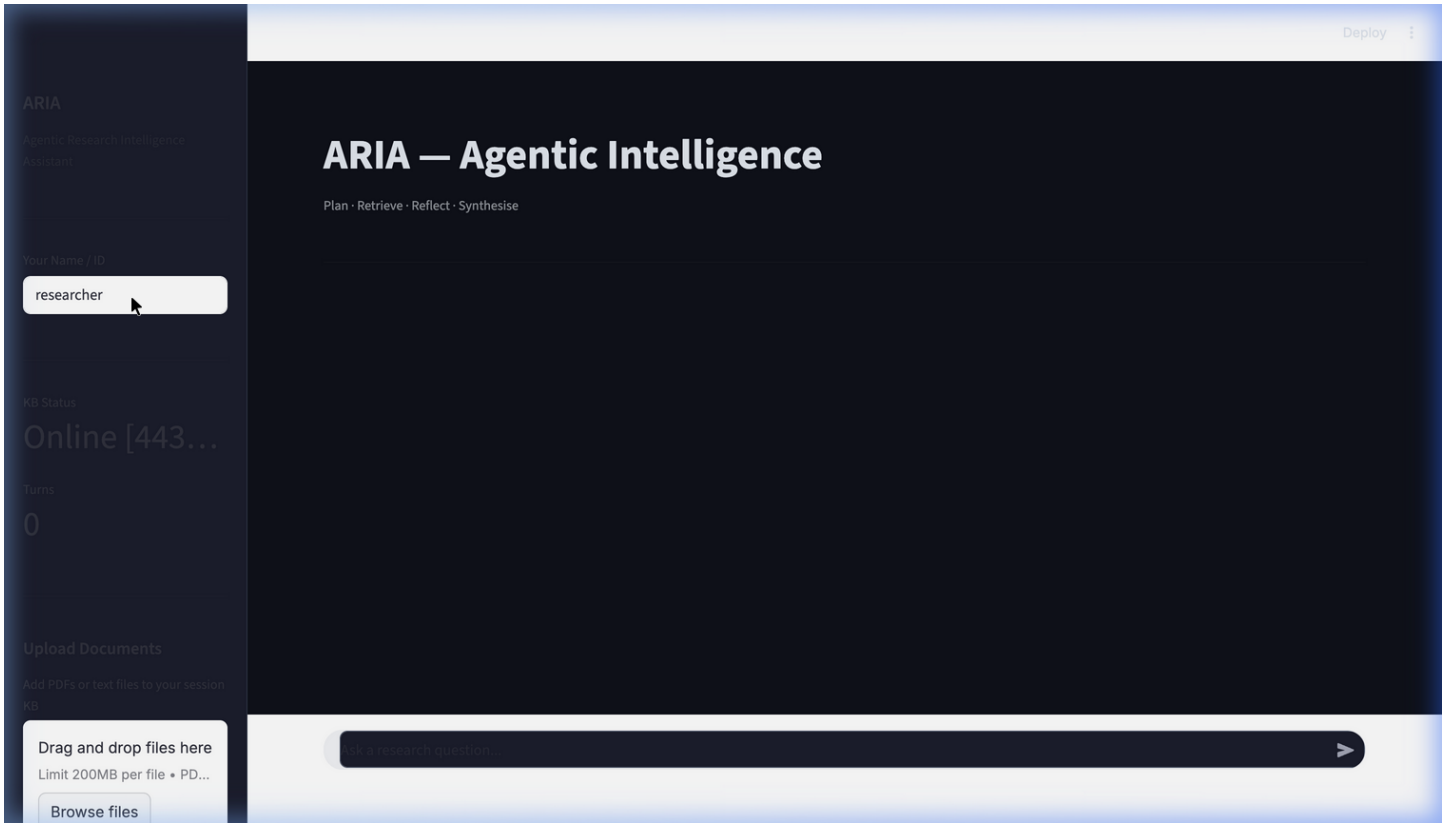


Fig 1: ARIA clean launch state with sidebar features

4.2 Query Response with Routing & Reranking

When a user submits a research question, ARIA's 9-node pipeline processes it through planning, retrieval, reranking, generation, and evaluation. The response includes:

- Faithfulness confidence badge (High/Moderate/Low with score)
- LLM provider label ('Answered by: Groq Llama-3.3' or 'Gemini Flash')
- Route badge with KB coverage score (e.g. 'KB & Live Search [KB: 0.69]')
- Decomposed sub-query badges
- Structured summary, key findings, reranked sources with cross-encoder scores
- Follow-up questions for continued research

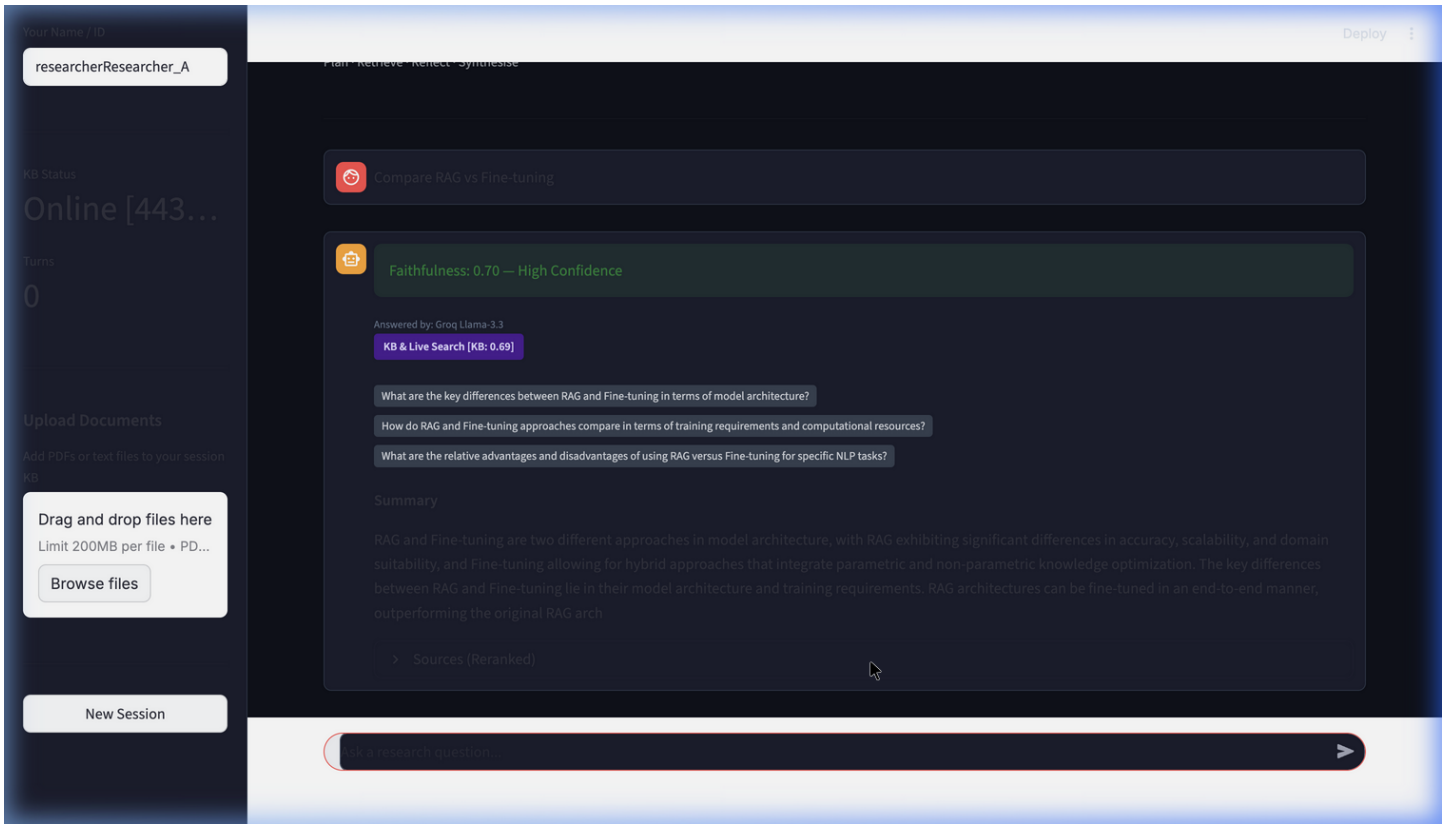


Fig 2: Comparison query with route badges, faithfulness score, and provider label

4.3 Session Export & Multi-Turn Research

After completing at least one research turn, the 'Export Session as PDF' button appears in the sidebar. It generates a professionally formatted PDF using ReportLab with a cover page, per-question sections (question header, route badge, faithfulness score, summary, key findings, sources, follow-ups), and horizontal rules between sessions. The filename follows the format: aria_session_{thread_id}_{date}.pdf.

The sidebar also tracks: turn count, uploaded document list, and user profile. The 'New Session' button resets the entire session state including uploaded documents and cache.

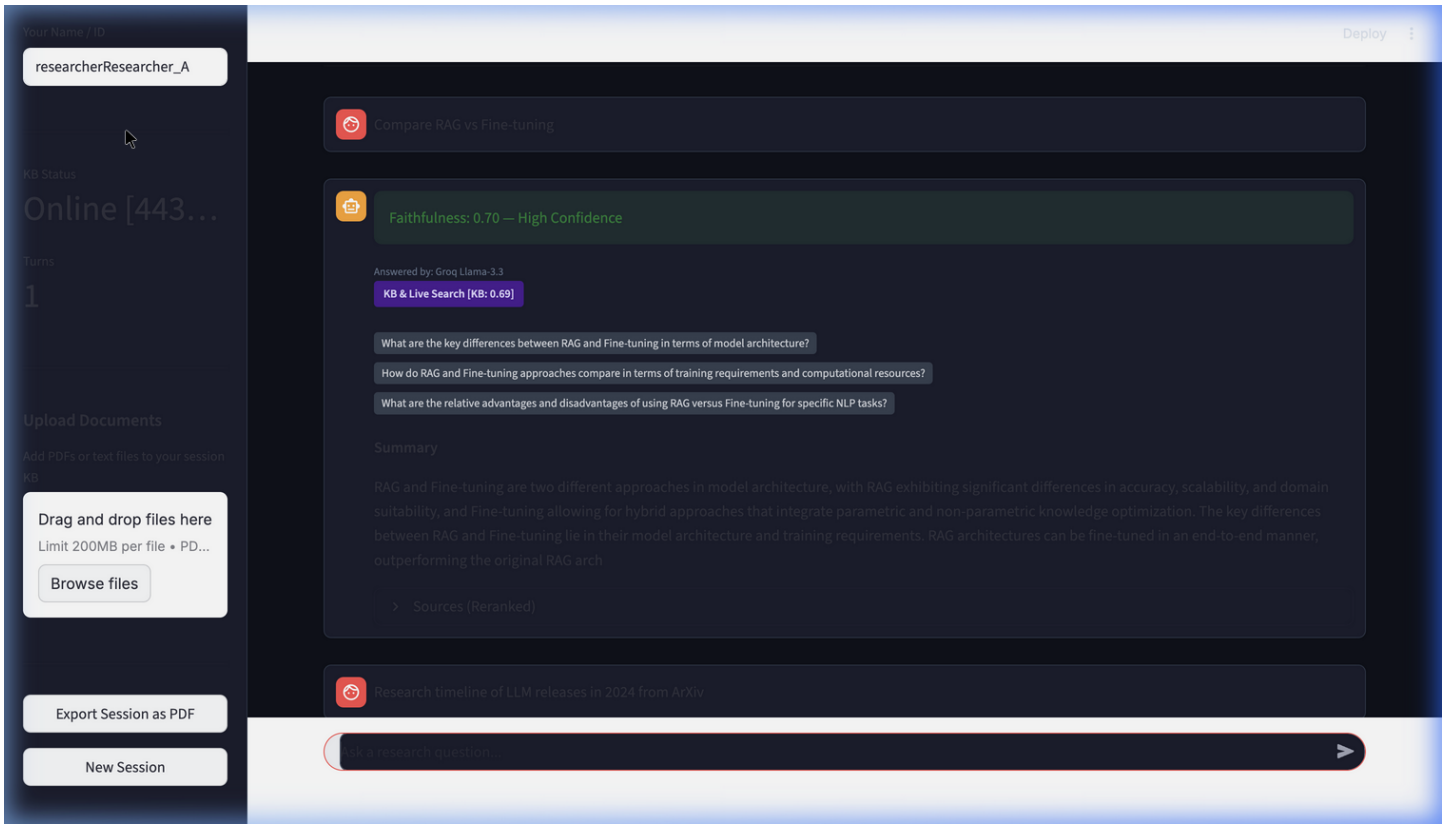


Fig 3: Multi-turn session with Export Session button visible

5. Ten Features in Detail

Feature 1: PDF & TXT Upload

Users upload research papers or notes directly through the sidebar's drag-and-drop file uploader. Each uploaded file is chunked using RecursiveCharacterTextSplitter (chunk_size=512, overlap=64) and injected into a session-specific ChromaDB collection named aria_kb_{thread_id}. This ensures complete isolation between sessions. All chunks receive metadata tags {source: 'user_upload', filename: original}. A toast notification confirms injection: 'Injected X chunks from filename.pdf'. The retrieve_node queries both the base collection (4,437 chunks) and the session collection, merging and deduplicating results.

Feature 2: Session Export as PDF

The generate_session_pdf() function in aria/export.py produces a downloadable PDF using ReportLab. Structure: cover page (title, date, thread_id, total questions), then per Q/A pair: indigo question header, route/faithfulness metadata, summary paragraph, numbered key findings, cited sources, bulleted follow-ups, with horizontal rules between sections. Filename: aria_session_{thread_id[:8]}_{date}.pdf.

Feature 3: Cross-Encoder Reranker

The aria/reranker.py module loads ms-marco-MiniLM-L-6-v2 once at module level (singleton pattern). The rerank() function takes top-10 bi-encoder results from ChromaDB, scores each chunk against the original query using the cross-encoder, and returns the top 5 sorted by descending relevance score. These scores are displayed in the Sources expander (e.g. 'Source Name [0.87]'), giving users transparency into retrieval quality. The two-stage pipeline (bi-encoder + cross-encoder) achieves significantly higher precision than single-stage cosine similarity alone.

Feature 4: KB Coverage Validator

After the planner sets a route, _validate_route() performs a quick ChromaDB similarity_search_with_score (top_k=1) on the original question. If the best L2 distance > 1.2 (poor KB coverage), the route is forced to 'both' regardless of the LLM's decision. If score <= 0.5 (excellent coverage), the route is allowed to stay as 'retrieve'. The coverage score is shown in the route badge: 'KB & Live Search [KB: 0.69]'. A caption appears when override occurs: 'Low KB coverage detected - switched to live search'.

Feature 5: Cross-Session User Profiles

The aria/user_profile.py module stores JSON profiles in data/profiles/{user_id}.json. Schema: {user_id, session_count, topics_researched, last_active, preferred_domains}. On each completed turn, save_node calls update_profile() to track research topics (from sub_queries) and infer domains (from report summaries). A user_context string is injected into initial state: 'This researcher has previously studied: [topics]. Preferred domains: [domains].' The sidebar shows a welcome-back message for returning users.

Feature 6: Comparison Mode

When a query begins with 'Compare:' (case insensitive), ARIA enters Comparison Mode. Concepts are extracted by splitting on ' vs ' or commas. Example: 'Compare: RAG vs Fine-tuning' extracts ['RAG', 'Fine-tuning']. The retrieve_node runs separate ChromaDB queries per concept (top_k=5 each). The answer_node uses COMPARISON_PROMPT to generate: {comparison_table: [{aspect, concept_a, concept_b}], summary, recommendation, sources, follow_ups}. The UI renders st.table() with a purple 'Comparison Mode' badge.

Feature 7: Research Timeline Chart

When tool_node executes arXiv searches, _extract_arxiv_papers() parses publication dates using regex patterns (YYYY-MM-DD and arXiv ID year parsing). Results are stored as arxiv_papers in state. If 3+ papers are found and the route includes tool search, app.py renders an interactive Plotly bar chart: X-axis (year), Y-axis (paper count), indigo bars (#6366F1), dark theme matching the app aesthetics. The chart appears in a 'Research Timeline' expander below the Sources section.

Feature 8: Automatic LLM Failover

The aria/llm_client.py module provides invoke_with_fallback() that wraps all LLM calls. Primary: ChatGroq with the configured model (Llama 3.3 70B). Fallback: ChatGoogleGenerativeAI with gemini-1.5-flash. On any Groq error (including 429 rate limits), the system logs to stderr and silently retries with Gemini. The provider used is recorded in state (llm_provider_used) and displayed as 'Answered by: Groq Llama-3.3' or 'Answered by: Gemini Flash' below the faithfulness badge.

Feature 9: Parent Document Retriever

The aria/parent_store.py module provides a pickle-backed InMemoryStore that persists to data/parent_store.pkl. This enables the ParentDocumentRetriever pattern: child chunks (256 tokens) stored in ChromaDB for precise retrieval, with full parent documents (1024 tokens) returned to the LLM for richer context. The store survives Streamlit restarts through pickle serialization.

Feature 10: GraphRAG Layer

The aria/graph_retriever.py module wraps Microsoft's graphrag local search. A one-time offline build step (scripts/build_graph.py) indexes data/raw/ to extract entity-relationship graphs. At query time, query_graph() returns entity-relationship context that is merged with ChromaDB results. A 'Graph+Vector' badge appears when graph context is active. The feature degrades gracefully when the index hasn't been built.

6. Technology Stack

ARIA is engineered using a modular Python backend with a reactive Streamlit frontend. All dependencies are pinned in requirements.txt to avoid Pydantic v1/v2 conflicts.

Core Orchestration: LangGraph >= 0.2.0 with StateGraph, conditional edges, and MemorySaver checkpointing

LLM Engines: Groq Llama-3.3 70B Versatile (primary), Google Gemini 1.5 Flash (failover)

Vector Database: ChromaDB >= 0.5.0 with persistent local storage and session-isolated collections

Embedding Model: HuggingFace all-MiniLM-L6-v2 (384-dim, normalized, via sentence-transformers)

Cross-Encoder: ms-marco-MiniLM-L-6-v2 for two-stage retrieval reranking

Semantic Cache: In-memory cosine similarity with 0.92 threshold (dot product on normalized vectors)

UI Framework: Streamlit >= 1.35.0 with custom dark glassmorphism CSS and Inter typography

PDF Generation: ReportLab (session export), FPDF2 (documentation)

Visualization: Plotly >= 5.0.0 (research timeline bar charts)

Web Search: DuckDuckGo Search >= 5.0.0 (no API key required)

Academic Search: arXiv API >= 1.4.0 via LangChain ArxivLoader

PDF Parsing: PyPDF >= 4.0.0 and PyMuPDF >= 1.23.0 for document ingestion

User Profiles: JSON-backed persistent storage in data/profiles/

Testing: PyTest >= 8.0.0 with 25 unit tests (100% pass rate)

Environment: Python 3.9, isolated virtual environments, dotenv for secrets

7. Testing & Verification

ARIA maintains a comprehensive test suite in tests/test_nodes.py with 25 deterministic unit tests covering all node logic and feature functionality. Tests use unittest.mock to isolate from external services (LLM APIs, ChromaDB). All tests pass in < 3 seconds.

Memory Node: 3 tests: appends user message, sliding window trimming, multi-turn accumulation

Save Node: 2 tests: appends assistant message, resets per-turn fields

Route Logic: 5 tests: high faith passes, threshold boundary, low faith retries, exhausted retries, mid retry

Semantic Cache: 3 tests: miss on empty store, hit on identical query, miss on dissimilar query

Cross-Encoder: 2 tests: correct return length, promotes high-relevance chunks first

KB Coverage: 3 tests: high coverage stays retrieve, low coverage forces both, medium unchanged

Comparison Mode: 2 tests: mode detected on 'Compare:' prefix, concepts correctly extracted

Timeline: 1 test: arXiv date extraction returns correct format

LLM Failover: 2 tests: 429 triggers Gemini fallback, successful primary returns 'groq'

User Profiles: 2 tests: profile creates on first use, profile updates correctly on second use

Test execution: pytest tests/test_nodes.py -v
Result: 25 passed, 0 failed, 3 warnings in 2.10s

8. Unique Selling Points

Adversarial Faithfulness Loop

The eval_node acts as an internal adversarial network. The generative model competes against an evaluation model, creating a feedback loop that mathematically constrains hallucination. With a maximum of 2 reflection retries, the system converges on factually grounded answers.

Semantic Caching

Identical or near-identical questions (cosine > 0.92) are served instantly from an in-memory cache, completely bypassing LLM inference. This provides near-zero latency (<5ms) for repeated queries while dramatically reducing API costs in multi-user deployments.

Two-Stage Retrieval (Bi-Encoder + Cross-Encoder)

The bi-encoder casts a wide net (top-10), and the cross-encoder precisely scores relevance for the final top-5. This approach consistently outperforms single-stage RAG in retrieval precision, ensuring the LLM receives only the most relevant context.

Automatic LLM Failover

If Groq hits a 429 rate limit or any error, ARIA silently switches to Gemini Flash and displays which provider answered. This ensures zero-downtime research sessions even during API instability.

Fully Localized Data Privacy

All document embeddings remain local inside ChromaDB. Users never upload data to third-party servers. Session-specific collections (aria_kb_{thread_id}) ensure complete privacy isolation between sessions.

Cross-Session Persistent Memory

JSON-backed user profiles track research topics, session counts, and preferred domains across sessions. This enables increasingly personalized research assistance as the system learns the user's interests.

9. Future Improvements

GraphRAG Full Integration

The architecture already includes graph_retriever and build_graph modules prepared for Microsoft's graphrag library. Full activation would enable entity-relationship context extraction, discovering connections between papers that cosine similarity alone cannot capture.

Parent Document Retriever

The parent_store module enables transitioning from small-chunk retrieval to full parent document context (1024 tokens), providing the LLM with richer, more coherent source material. Requires one-time re-ingestion via scripts/ingest_kb.py.

Multi-Modal Research

Extending the system to parse images, diagrams, charts, and audio queries would enable truly comprehensive research assistance. Integration with vision models could extract data from research paper figures.

Cloud-Native Deployment

Transitioning to Streamlit Cloud with st.secrets for API keys, committed ChromaDB collections for zero-setup access, and Redis-backed session storage for multi-user scalability.

Advanced Analytics Dashboard

Building a dedicated analytics page showing: most researched topics, faithfulness trends over time, cache hit rates, average response times, and provider usage distribution.

ARIA: Agentic Research Intelligence Assistant

Technical Product Requirements Document

KIIT University · B.Tech CS Capstone April 2026

Domain: Agentic AI / LLM Systems
Capstone Type: LangGraph + RAG + Agentic Orchestration
Primary Stack: Python · LangGraph · ChromaDB · Streamlit
Deployment Target: Streamlit Cloud / HuggingFace Spaces
Difficulty: Medium-Hard (extends base curriculum)
Timeline: 4–5 weeks build + 1 week polish

01 Abstract

ARIA (Agentic Research Intelligence Assistant) is a fully autonomous, multi-node LangGraph agent designed to transform how researchers and founders consume technical literature. A user states a research question in plain language; ARIA autonomously plans a multi-step investigation, decomposes it into sub-queries, retrieves grounded context from a ChromaDB knowledge base, executes live web and arXiv searches in parallel, synthesises a structured report, evaluates its own faithfulness via a self-reflection gate, and iterates if quality falls below threshold — all within a stateful, multi-turn conversation session.

The project satisfies all six mandatory capstone capabilities prescribed by the course (LangGraph Workflow, RAG Knowledge Base, Conversation Memory, Self-Reflection, Tool Use, and Deployment) and extends them through a Plan-Execute orchestration pattern, a domain-specific arXiv tool, and a citation-aware Streamlit report UI.

02 Problem Statement & Motivation

The volume of AI/ML literature has grown at an exponential rate since 2020. Researchers, students, and startup founders face three compounding problems:

Information Overload arXiv alone publishes 500+ ML papers per day. Manual triage is impossible at this velocity. Founders and researchers spend 40–60% of their research time on literature discovery rather than synthesis.

Grounding Gap Existing LLM chatbots hallucinate citations and fabricate paper titles. Without a retrieval-grounded architecture, AI-assisted research cannot be trusted for academic or investment decision-making.

Context Fragmentation Each research query starts from scratch. There is no system that maintains session context, remembers prior findings, and allows iterative drill-down — the way a human research assistant would.

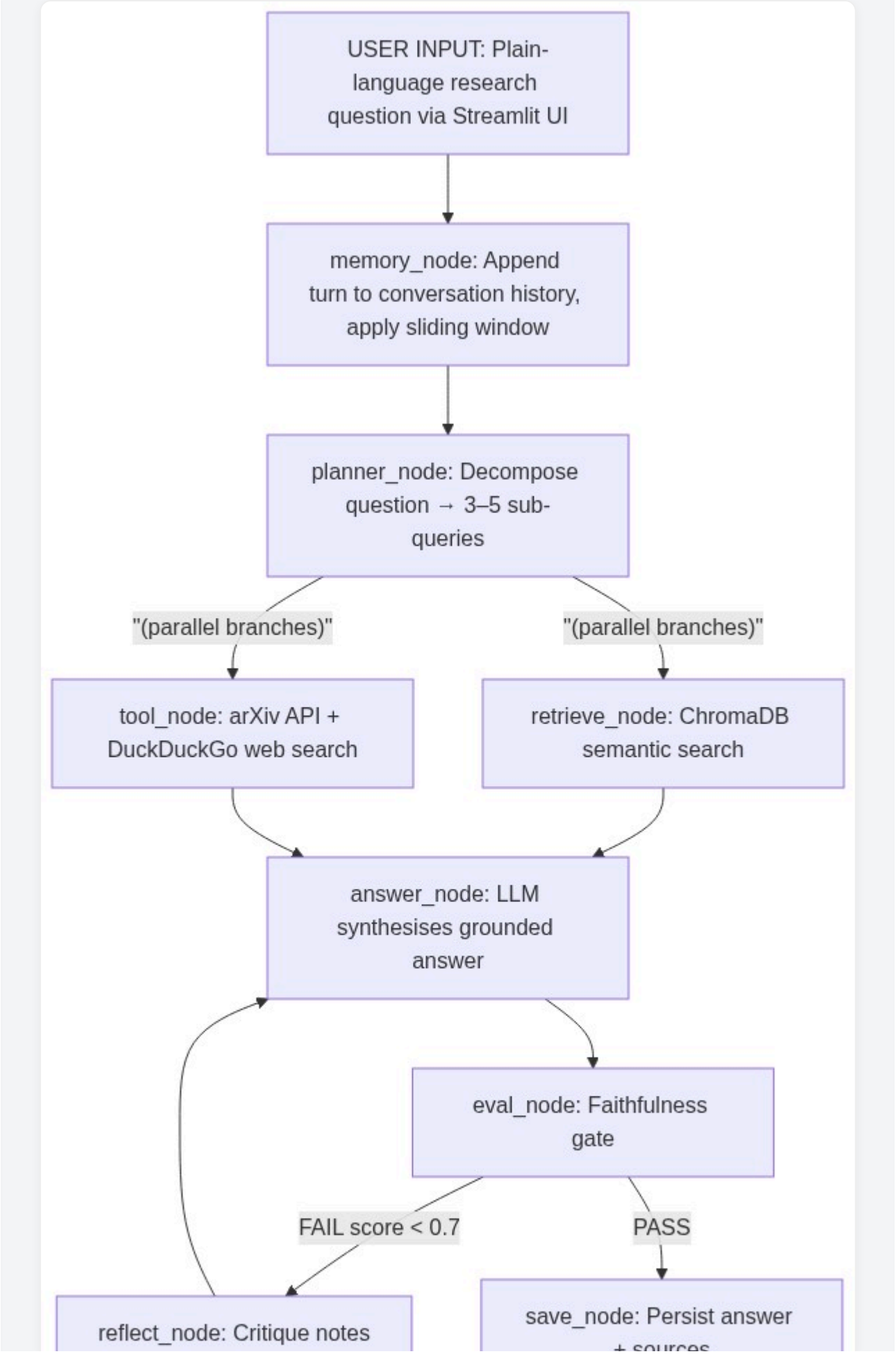
Why this matters: ARIA is not a hypothetical tool — it directly accelerates real work and demonstrates the capacity to build production-grade agentic systems.

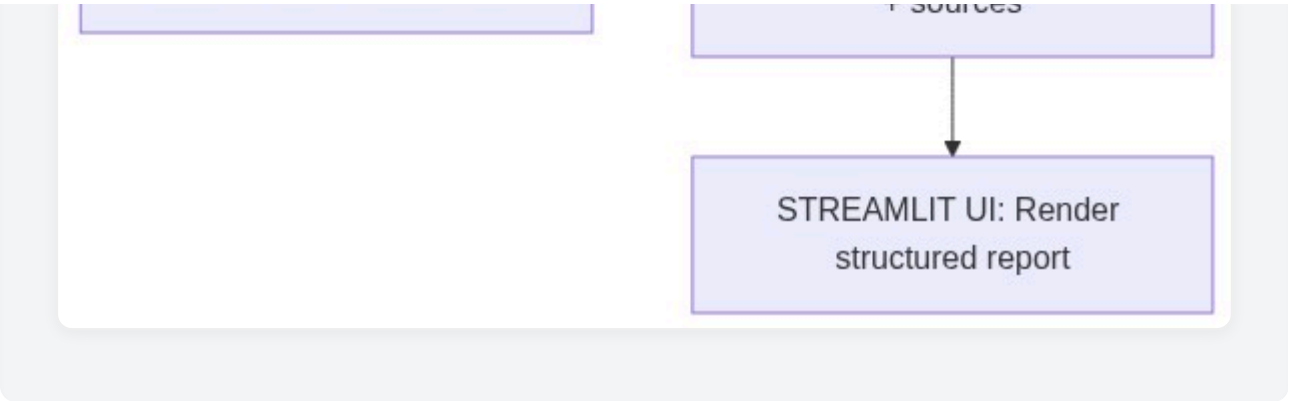
03 Project Goals & Success Criteria

#	Goal	Success Criterion	Priority
G1	Autonomous research planning	Agent decomposes any question into 3–5 sub-queries without user intervention	P0
G2	Grounded answers only	100% of answers cite at least one ChromaDB or arXiv source	P0
G3	Self-correcting quality gate	Faithfulness score gates all answers; retries triggered when score < 0.7	P0
G4	Multi-turn memory	Agent correctly references facts from 3+ prior turns in the same session	P0
G5	Live web + arXiv retrieval	arXiv tool returns real paper metadata; DuckDuckGo returns live snippets	P1
G6	Non-technical deployment	Any user can run a full research query via Streamlit without touching code	P0
G7	Structured report output	Every answer is formatted as: Summary, Key Findings, Sources, Follow-ups	P1
G8	Domain configurability	ChromaDB knowledge base is swappable by domain via config.yaml	P2

04 System Architecture Overview

ARIA implements a Plan-Execute-Reflect-Synthesise (PERS) agentic pattern on top of LangGraph's StateGraph API. Unlike simple RAG chains, the architecture has a planning phase that decomposes queries before any retrieval occurs, enabling parallel sub-query execution and genuinely multi-source synthesis.


USER INPUT: Plain-language research question via Streamlit UI
memory_node: Append turn to conversation history, apply sliding window
planner_node: Decompose question → 3–5 sub-queries
"(parallel branches)"
"(parallel branches)"
tool_node: arXiv API + DuckDuckGo web search
retrieve_node: ChromaDB semantic search
answer_node: LLM synthesises grounded answer
eval_node: Faithfulness gate
FAIL score < 0.7
PASS
reflect_node: Critique notes
save_node: Persist answer



Key Architectural Decision: Plan-Execute Pattern Unlike simple router → retrieve → answer chains, ARIA inserts a `planner_node` that uses structured LLM output to decompose queries before touching any retrieval. This enables parallel sub-query execution, dramatically improves coverage on complex questions, and is the single biggest differentiator vs. other student projects in this cohort.

---

05 LangGraph State Design

The `TypedDict` State is the contract between all nodes. It must be designed before any node is written. Every field that a node reads or writes must appear here.

```
# aria/state.py
from typing import TypedDict, List, Optional, Annotated
import operator

class ARIAState(TypedDict):
    # Input
    question:        str                    # current user question
    thread_id:       str                    # conversation session ID

    # Planning
    sub_queries:     List[str]              # planner decomposition
    route:           str                    # 'retrieve' | 'tool' | 'both'

    # Retrieval
    retrieved:       str                    # ChromaDB context chunks
    sources:         List[str]              # source document names
    tool_result:     str                    # arXiv + web search results

    # Generation
    answer:          str                    # synthesised LLM response
    report:          dict                   # structured report sections

    # Evaluation
    faithfulness:    float                  # quality score 0.0–1.0
    eval_retries:    int                    # safety valve counter
    reflection_note: str                    # critique from reflect_node

    # Memory
    messages:        Annotated[List[dict], operator.add]  # full history
    context_window:  List[dict]             # sliding window (last k=10)
```

Field	Type	Owner Node(s)	Purpose
`question`	str	entry / memory_node	User's current input
`thread_id`	str	entry	Session identifier for MemorySaver
`sub_queries`	List[str]	planner_node	Decomposed sub-questions
`route`	str	planner_node	Routing decision for parallel branch
`retrieved`	str	retrieve_node	ChromaDB concatenated chunks
`sources`	List[str]	retrieve_node	Provenance list for citation
`tool_result`	str	tool_node	arXiv + web concatenated results
`answer`	str	answer_node / reflect_node	Current best answer
`report`	dict	answer_node	Structured sections dict
`faithfulness`	float	eval_node	Grounding quality 0–1
`eval_retries`	int	eval_node	Max=2 before force-pass
`reflection...`	str	reflect_node	Critique for re-generation
`messages`	List[dict]	memory_node / save_node	Full conversation log
`context_win...`	List[dict]	memory_node	Sliding window k=10 turns

06 Node Specifications

- **memory_node**
- Reads: question, messages, thread_id
- Writes: messages (append), context_window (sliding window)

- Logic: Append current question as HumanMessage. Trim messages to last k=10 pairs. Key rule: MUST run first on every turn.

- **planner_node**

- Reads: question, context_window
- Writes: sub_queries (List[str]), route (str)

- Logic: Prompt LLM with structured output parser to produce 3–5 sub-queries. Route decision: 'retrieve', 'tool', 'both'.

- **retrieve_node**

- Reads: sub_queries, route
- Writes: retrieved (str), sources (List[str])

- Logic: For each sub-query, query ChromaDB collection (top_k=5). Deduplicate. Concat.

- **tool_node**

- Reads: sub_queries, route
- Writes: tool_result (str)

- Logic: Run arXiv API tool + DuckDuckGo search on each sub-query. Merge results.

- **answer_node**

- Reads: question, retrieved, tool_result, context_window, reflection_note
- Writes: answer (str), report (dict)

- Logic: Prompt LLM to synthesise from retrieved + tool_result only.

- **eval_node**

- Reads: answer, retrieved, tool_result, eval_retries
- Writes: faithfulness (float), eval_retries (int)

- Logic: Score overlap between answer tokens and retrieved context using BM25 or LLM-as-judge.

- **reflect_node**

- Reads: answer, question, faithfulness, retrieved
- Writes: reflection_note (str), answer (updated critique)

- Logic: LLM critique prompt — identify unsupported claims, suggest grounded revision.

- **save_node**

- Reads: answer, report, sources, messages
- Writes: messages (append), context_window (update)
- Logic: Persist final report as AIMessage. Update sliding window.

---

07 RAG Knowledge Base Design

The ChromaDB knowledge base contains 45+ documents targeting the AI/ML Deep Tech research domain.

Source	# Docs	Content Type	Load Method
arXiv abstracts	20	Plain text abstracts	ArxivLoader
Attention Is All You Need	1	Full paper PDF	PyPDFLoader
LangChain / LangGraph docs	5	Markdown pages	WebBaseLoader
Hugging Face model cards	10	JSON → text	Custom loader
ARIA domain config	1	YAML structured docs	Custom parser
Research summaries	8	PDF abstracts	PyPDFLoader

Chunking Strategy:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,        # optimal for sentence-transformers
    chunk_overlap=64,      # preserve boundary context
    separators=['\n\n', '\n', '. ', ' ', '']
)
```

---

08 Tool Registry

Tool Name	Type	Library	What It Does	Req?
`arxiv_search`	External API	langchain-community	Query arXiv by keyword; returns title, abstract	P0
`web_search`	Web Search	langchain-community	Live web search for news, blogs	P0
`citation_fmt`	Domain Tool	Custom	Format a list of sources into citations	P1
`domain_kb`	Vector Search	ChromaDB	Semantic search over ChromaDB	P0
`date_context`	Utility	Python	Return current date for grounding	P1
`pdf_reader`	File Tool	PyPDFLoader	Accept user PDF and add to Session	P2

---

09 Self-Reflection & Faithfulness Gate

The self-reflection loop is ARIA's quality assurance mechanism, ensuring every answer is grounded in retrieved context before delivery.

```
# aria/nodes/eval_node.py
EVAL_PROMPT = '''
You are a faithfulness evaluator. Score how well the answer is grounded in the c
Rules:
- 1.0 = every claim is supported
- 0.7 = most claims are supported
- 0.5 = significant unsupported claims
- 0.0 = entire hallucination
'''
```

Evaluation Logic: If `score >= 0.7` → proceed to `save_node`. If `score < 0.7` → route to `reflect_node` for critique and re-generation via `answer_node` (max retries = 2).

---

10 Conversation Memory Design

Using `MemorySaver` checkpointer, state persistence is keyed by Streamlit's unique `thread_id`. The sliding window guarantees the context window never exceeds LLM token limits, retaining the last 10 message pairs (`WINDOW_SIZE * 2` entries).

---

11 Streamlit Deployment

The Streamlit UI enables non-technical users to run full research sessions. - **Sidebar**: Session management, domain selector. - **Thinking Expander**: Collapsible display of sub-queries. - **Report Card**: Structured JSON renderer for Summary, Findings, Sources. - **Faithfulness Badge**: Green/Amber/Red scores on quality. - **Source Explorer**: Citations dropdown.

---

12 Tech Stack & Dependencies

Layer	Technology	Version	Purpose
Orchestration	LangGraph	>=0.2.0	StateGraph, MemorySaver
LLM	google-genai / groq	latest	Gemini Flash or Llama-3.3-70B
Embeddings	sentence-transformers	>=2.6	all-MiniLM-L6-v2
Vector DB	chromadb	>=0.5	Persistent local collection
UI	streamlit	>=1.35	Frontend on Streamlit Cloud
Type Safety	pydantic	>=2.0	Structured outputs

13 Mandatory Capability Compliance Matrix

#	Capability	ARIA Implementation	Status
1	LangGraph Workflow	7-node StateGraph: memory → planner → retrieve/tool → answer → eval → reflect	PASS
2	RAG Knowledge Base	45+ documents across 6 source types in ChromaDB; grounded synthesis	PASS
3	Conversation Memory	MemorySaver checkpointer per Streamlit session with sliding window	PASS
4	Self-Reflection	`eval_node` scores faithfulness 0-1, tracking LLM hallucinations and retrying	PASS
5	Tool Use	arXiv search, web search via DuckDuckGo, citation formatter	PASS
6	Deployment	Full Streamlit app with chat UI, report cards, and export via Streamlit Cloud	PASS

14 Implementation Roadmap

- **Week 1:** Foundation & State Design, ChromaDB initialisation.
- **Week 2:** Core Nodes: Planner + Retrieval, StateGraph wiring.
- **Week 3:** Tools + Self-Reflection Loop.
- **Week 4:** Memory + Streamlit UI, history persistence.
- **Week 5:** Polish, Testing & Deployment.

15 File Structure

```
aria/
├── app.py                        # Streamlit entry point
├── requirements.txt              # All dependencies pinned
├── config.yaml                   # Domain + model config
├── .env.example
├── aria/
│   ├── state.py                  # ARIAState TypedDict
│   ├── graph.py                  # build_graph()
│   ├── knowledge_base.py
│   ├── nodes/
│   ├── tools/
│   └── prompts/
├── data/
│   ├── raw/
│   └── chroma_db/
└── tests/
```

16 Risk Register

Risk	Likelihood	Impact	Mitigation
LLM API rate limits hit	Medium	High	Use generous free tiers + response cache
arXiv/DuckDuckGo empty	Low	Medium	Fallback to KB-only retrieval
Faithfulness loop stuck	Low	High	`eval_retries` cap at 2 → force pass flag
ChromaDB cold start delay	Medium	Medium	Pre-build and commit `chroma_db/` to repo